

From Words to Walk Cycles

Learned 2D Generation versus Disposable 3D Proxies for Eight-Direction Pixel-Art Sprites

Pablo Bustamante
Independent researcher
Chile



Figure 1: Fourteen characters generated from one-line ideas, front view, covering twelve body plans. Each one exists as an eight-direction sheet with a four-frame walk cycle, produced without any sprite training data.

ABSTRACT

Top-down pixel-art games and RPGs need every character drawn from eight directions with a walk cycle, and the eight drawings must agree with each other. None of the models we tried produces such sheets for new characters consistently. We report two routes toward it, built in the same project on two machines. Route A, on an NVIDIA RTX 3090, trained three generations of a sprite-sheet generator on 2,984 community sheets: predicting pixel roles instead of colors made creatures legible, but with 2,603 unique designs the model memorized after about 30,000 steps, and its best results came from reassembling real parts. Route B, on an Apple M5, routes a one-line idea through a disposable 3D proxy: a local language model rewrites the idea for one of thirteen body plans, FLUX.2 [klein] draws a concept, Pixa3D lifts it to a mesh, Blender rigs and renders eight directions with rule-based rigs, and a 3D-aware pixelization reduces each frame using depth, projected eyes and object-space limb thickness. Route B won. It needs no sprite data or training, renders eight true directions, and takes about four and a half minutes per creature. On 14 creatures its pixelization cuts isolated-pixel noise from 8.3% to 3.4% and keeps 92% of visible eyes legible (53% for a naive downscale). We argue that Route B, combined with palette donation and mirroring, can grow the per-body-plan datasets that Route A lacked.

KEYWORDS

pixel art, sprite generation, top-down games, single-image 3D generation, discrete diffusion, procedural animation

1 INTRODUCTION

A character in a top-down pixel-art game or RPG is stored as a sprite sheet: eight directions (down, down-right, right, up-right, up, up-left, left, down-left), each with a short walk cycle. Drawing one good sprite is the easy part; drawing eight that describe the same body is what costs artists time.

Generative models do not solve this yet. In this project we tried SDXL [23] with a pixel-art LoRA, FLUX.2 [klein] [6], Qwen-Image [25] and two sprite generators of our own. Image generators draw convincing single sprites but do not keep a character’s shape when it turns; Qwen-Image produced coherent eight-direction sheets from a layout guide but copied the guide’s identity; multi-view diffusion [18, 27, 31] is consistent for renders and photographs, but at 40–52 pixels any disagreement between views reads as a different character. We found no single model that does the whole job consistently. A pipeline does.

We followed two routes, on two machines, and report both:

- **Route A, learned 2D generation** (NVIDIA RTX 3090). Three generations of a sheet generator trained on community sprite sheets, an agent supervising training, and a part-assembly system on top (Section 3).
- **Route B, disposable 3D proxy** (Apple M5). Text to concept to mesh to rig to eight rendered directions to 3D-aware pixelization (Sections 4–6).

Route B won (Section 8). Our contributions are a local pipeline from a one-line idea to eight-direction walk cycles; morphology-aware animation for thirteen body plans with one gait rule; a pixelization stage that uses the 3D data a 2D downscaler lacks; measurements on 14 creatures; an account of why a learned generator

falls short on today’s data; and a path by which Route B can produce the data it needs.

2 RELATED WORK

Pixel-art abstraction. Gerstner et al. [10] jointly optimize a low-resolution grid and a palette; Han et al. [11] learn pixelization without paired data; SD- π XL [5] distills a diffusion prior into quantized images; Kopf and Lischinski [14] solve the inverse problem. All work from one 2D image. Our input is a render with depth and known 3D feature positions, and 32 frames must share one palette.

Single-image and multi-view 3D. Zero-1-to-3 [18], Zero123++ [27] and SV3D [31] synthesize novel views; TRELLIS [32] and Hunyuan3D 2.0 [28] produce textured meshes through structured latents and flow matching [17]. We use Pixa3D through the open Image-to-3D Lab [4, 26], which conditions on pixel-aligned features and favors flat-colored character art.

Rigging and animation. Pinocchio [2] embeds a template skeleton; UniRig [33] predicts skeletons for diverse topologies. We need only four frames of walking at sprite scale, which geometric rules handle across body plans no template covers.

Discrete diffusion and prompt rewriting. Route A follows masked discrete diffusion [1, 8]. Route B rewrites prompts before image generation [3, 12], here to translate non-English ideas and to state the body plan the rig depends on.

3D for 2D in games. *Dead Cells* [30] and *Guilty Gear Xrd* [19] render hand-built 3D as 2D. Route B makes every step from text automatic.

3 ROUTE A: LEARNED 2D SHEET GENERATION

Route A ran on a PC with an NVIDIA RTX 3090 (24 GB, power-limited to 300 W), a Ryzen 9 5900X and 32 GB of RAM, between 28 September and 2 October 2026, for about 44 GPU hours.

Data. The PMD Sprite Repository [22], a community collection of eight-direction sheets licensed CC BY-NC 4.0, gave 2,984 sheets of 908 species, cropped to 48×48 with Walk and Idle resampled to four frames each. The model generates five directions; the three left-facing ones are mirrored, which also serves as data augmentation.

Roles instead of colors. Each pixel is assigned a role: transparent, outline, or one of three luminance tones within one of four hue regions ordered by area (14 roles). Roles reconstruct the sprites with a mean error of 4.9/255 and separate shape from color, so real palettes are applied afterwards. Collapsing recolors leaves 2,603 unique designs.

Three generations. Table 1 and Figure 2 summarize them. Diffusion on RGBA produced confetti. Masked discrete diffusion over roles (r2) produced recognizable creatures; balancing the training set by species cut copies of training species from 46% to 0%, and in a gallery of 100 quadrupeds about half were good, 30% acceptable and 20% weak. r3 added a full-resolution level and a 19-role scheme, and a vision-capable Claude Opus agent reviewed candidates every 1,000 steps and adjusted hyperparameters during training: of about

Table 1: Route A: three generations of a sheet generator and the part-assembly system built on the last one.

Run	Idea	GPU h	Outcome
r1	Continuous diffusion on RGBA	0.6	Confetti; stopped at 5,000 steps
r2	Masked discrete diffusion over 14 roles, 33.1 M params	10.3	Legible creatures; best at 31,000 steps
r3	r2 + a 48×48 level, 19 roles, an agent supervising training, 33.9 M params	33.2	Cleaner silhouettes; best at 44,500 steps
Parts	Real parts recombined by rules, r3 for seams	–	5 of 5 approved in the last two rounds

60 changes, only the learning rate showed a clear effect. It was a good reviewer and a weak optimizer.

The ceiling is data. In both r2 and r3 the loss on held-out species reached its minimum early (27,500–35,500 steps for r3) and only rose afterwards; the held-out to training ratio grew from 1.48 at step 10,000 to 4.4 at step 200,000, by which point some samples resembled existing species. With 2,603 designs, the model memorizes before it masters fine detail.

Part assembly. The best visual results came from recombining real sprites: parts detected by Claude Opus (an open vision-language model knew which parts existed but not where), pasted per view with impostors and an approximate 3D skeleton, with r3 filling seams. The last two rounds approved 5 of 5 mixes. Reconstructing geometry from 48-pixel sprites failed; a visual hull of the eight real views, with camera elevation self-calibrated to about 30° (IoU 0.79), worked for synthesizing intermediate angles while keeping the original drawings. That finding, 3D as scaffolding rather than as the source of the image, is the premise of Route B.

4 ROUTE B: DISPOSABLE 3D PROXIES

Route B ran entirely on an Apple M5 with 32 GB of unified memory (Figure 3).

4.1 Idea, prompt writer and concept

The user writes a short idea and picks one of thirteen body plans (Table 2). FLUX.2 [klein] understands Spanish poorly and, given two words, fills in the most common reading: “valkiria centaura” with a centaur body plan produced a plain horse (Figure 6). A local Qwen-family model [24] running in Ollama [20] therefore receives the idea and a one-line anatomy for the body plan, and writes a 40–60-word English description that keeps the iconic features of every part of the idea (clothing, armor, headgear, weapons) and names the main colors. The 27B-parameter writer is unloaded before FLUX loads, since both do not fit in memory at once.

A hidden template wraps the description: official-style creature character art, full body on plain white, flat two-tone cel shading and a silhouette designed to read at sprite scale, plus a pose that depends on the body plan (front view for bipeds, three-quarter view for quadrupeds, legs spread for insects, wings open for flyers). FLUX.2 [klein] 4B draws a 1024^2 concept in four steps (≈ 20 s).



Figure 2: Route A, three generations with the same three seeds. Each block is one generated sheet: five directions (rows) $\times$ four Walk and four Idle frames. Diffusion on RGBA (r1) gives confetti; predicting pixel roles (r2, r3) gives coherent creatures.

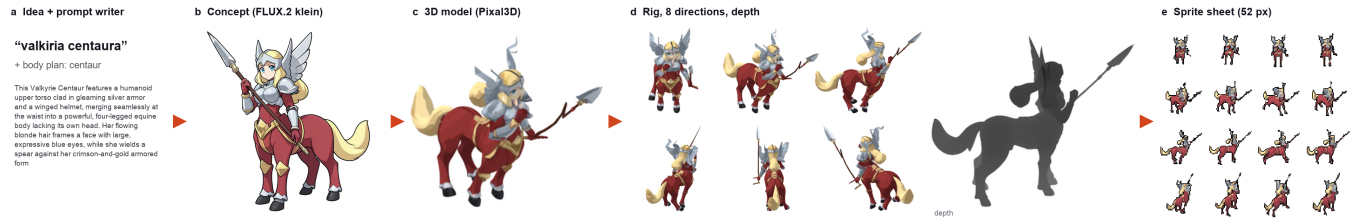


Figure 3: Route B on one example. (a) A two-word Spanish idea and a body plan are rewritten into an explicit English description. (b) FLUX.2 [klein] draws a full-body concept on white. (c) Pixal3D lifts it to a textured mesh. (d) Blender rigs the mesh and renders eight directions $\times$ four walk frames, plus depth. (e) The 3D-aware pixelization produces the sheet; four of eight rows shown.

An edit mode adds a requested change to an existing concept; our four-armed golem got its extra arms that way.

4.2 Disposable 3D proxy

Pixal3D receives the matted concept with a synthetic 20° front camera and runs three flows: sparse structure with classifier-free guidance [13] in a limited interval [16], shape at 512³ followed by a 1024³ cascade, and texture. The output, a GLB with about 300,000 faces, is rotated by 180° to face the camera; omitting that step made early results look “flipped” and cost us a long detour.

4.3 Rendering

Blender [7] renders with a fixed orthographic camera 30° above the horizon, the elevation Route A’s calibration found in the community sprites, and turns the creature in 45° steps. Each material becomes a three-band toon shader lit by a sun attached to the camera from the upper left; cast shadows are off, since a mushroom’s cap otherwise blacked out its face. The camera window covers all 32 deformed frames, so the creature keeps one scale, and every frame also writes a 16-bit depth image.

Table 2: Body plans. Each changes the pose requested from the image model, the anatomy given to the prompt writer, how the mesh is straightened, how it walks, and the sprite height in pixels.

Body plan	Animation	Heading	Height
biped	Leg rig (2) + arms that swing against the leg on their side	Perpendicular to the line between feet	40
quadruped	Leg rig (4), diagonal trot	Long axis of the four feet; sign from the face	48
multi-legged	Radial legs rotate 22° about their hip and lift 5%; tripod gait	Torso axis; none if round (ratio <1.25)	48
floating	Bob 3.5% of height, breathe 1.5%	–	40
blob	Squash 1.08×0.88, stretch 0.94×1.1 with a 4% hop	–	36
serpent	Lateral wave (amplitude 7%, wavelength 0.9) growing toward the tail	Long axis; head is the higher end	44
naga	Split at 30% height: tail waves, torso sways 3°, arms swing 20°	–	48
winged	Wings hinge beyond 28% of span: 35°, 0, –25°, 0; body rises on the down-stroke	Wingspan on x	48
centaur	Leg rig (4) + arms in the 55–95% height band	As quadruped	52
four-armed	Leg rig (2) + two arm pairs split by k -means on height	As biped	44
dragon	Leg rig (4) + back wings above 50% height (not yet evaluated)	As quadruped	52
fish	Float + wave from 40% of the length to the tail	Long axis; head is the thicker end	40
octopus	Tentacles found as thick radial legs, each with its own lateral wave	–	40

5 MORPHOLOGY-AWARE ANIMATION

Generated meshes arrive as one welded surface with no skeleton. Rather than predicting a rig, we exploit the body plan the user chose.

Legs by slicing. We cut the mesh with horizontal slices from 4% to 60% of its height in 1% steps and rasterize each into a 96^2 grid. A connected blob counts as a leg if it holds at least 0.4% of the grid and 8% of the slice, and its center lies within $0.8r$ of the vertical axis (r is the model radius); $0.55r$ dropped a fox’s diagonal foreleg, while no limit picks up tails. Two legs standing close merge into one blob; if a slice has three blobs and the largest is $1.6\times$ the median, we split it with 2-means. One bone per leg runs from the crotch to the foot, with weights from the nearest leg in plan view blended over 12% of the height above the crotch.

Insect legs by morphological opening. A beetle’s shell rim joins all legs at every height. We instead rasterize everything below 30% of the height top-down in a 192^2 grid and apply a morphological opening with a disk of 4.5% of the grid: the torso survives, the thin legs do not, and their difference splits into one component per leg. Octopus tentacles use a 10% disk.

One gait rule. Legs are split into two sides by the mean of their x coordinates and ranked $j = 0, 1, \dots$ from front to back on each side $s \in \{0, 1\}$. Each leg gets a phase

$$f = \begin{cases} +1 & (j + s) \bmod 2 = 0 \\ -1 & \text{otherwise.} \end{cases} \quad (1)$$

In walk frame $k \in \{0, 1, 2, 3\}$ a leg rotates about its hip by $-\alpha g_k f$ with $\alpha = 24^\circ$ and $g = (1, 0, -1, 0)$; the body rises 3% on frames 1 and 3 and sways 3° , and the swinging leg lifts 5%. Two legs alternate, four trot diagonally, six give the insect tripod and eight the alternating tetrapod of spiders.

Arms, wings and deforming bodies. Arms are vertex groups that stick out sideways from the torso within a height band; each swings $\pm 22^\circ$ against the leg on its side. A group extending front

to back more than 25% of the height is rejected: it was a mushroom’s cap. Bodies without a leg rig are animated by direct vertex deformation with closed-form motions (Table 2).

6 3D-AWARE PIXELIZATION

The 32 frames are reduced together to sprites 36–52 pixels tall.

Shared palette and concept color. A Kuwahara filter [15] with a window of one sprite pixel removes texture noise while keeping edges; k -means with $k = 12$ in OKLab [21] over all frames gives one palette, so nothing flickers as the creature walks. The palette is then pulled toward the concept: we rotate the dominant hue (up to $\pm 60^\circ$) and correct lightness and chroma per hue group (12 groups of 30° , neutrals excluded, lightness change clamped to $[-0.25, 0.35]$). A global correction had turned a red mushroom cap pink.

Thin limbs in object space. One sprite pixel measures

$$p = \frac{h \cos \theta + 2r \sin \theta}{H} \quad (2)$$

in world units, for model height h , radius r , camera elevation θ and sprite height H . A leg of diameter $d < 1.4p$ is inflated along vertex normals by $\min((1.4p - d)/2, 0.8p)$ before animating, which keeps it continuous in every direction and frame.

Detail-preserving block reduction. Each sprite pixel takes the most frequent palette color of its block, except that anything thinner than about 0.8 sprite pixels counts as detail and wins its block if it covers 18% of it. Isolated pixels and spikes are removed, and a pixel close in color to the dominant color of five of its eight neighbors takes that color.

Lines from depth. Inner lines are drawn where depth jumps by more than 0.03 of its range. The outline is two-tone: on parts one or two pixels wide only the shadow side is near-black.

Projected eyes. Eyes are detected once in a front render, tied by a ray cast to a mesh vertex, and projected into every animated frame. An eye is stamped only if it faces the camera and the depth pass

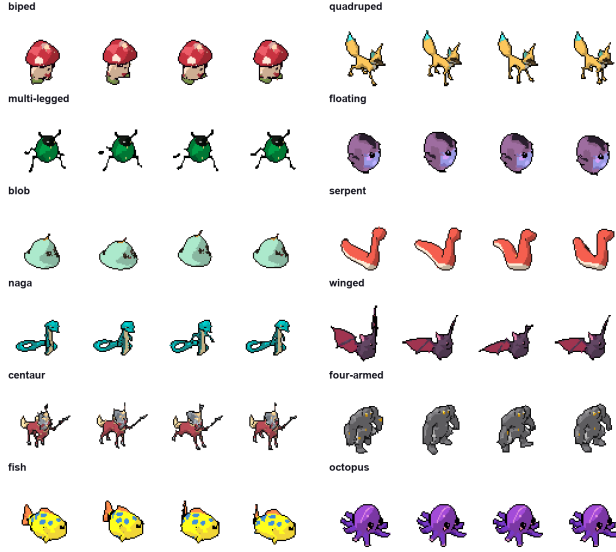


Figure 4: Route B walk cycles (four frames, facing down-right) for twelve body plans.

confirms nothing occludes it; if something covers it from above it peeks out just below.

7 RESULTS

We generated 14 creatures covering 12 of the 13 body plans with Route B (Figures 1 and 4); the dragon plan is implemented but not yet evaluated.

7.1 Ablation of the pixelization

We compare the full pixelization against a naive baseline (area downscale to the same window, then a shared 12-color k -means palette in OKLab) and against the full pipeline with one component removed, on the same renders. **Noise** is the percentage of opaque pixels with no 8-neighbor of the same color; **fragments** counts detached silhouette pieces per frame; **eyes** is the percentage of the 386 eye instances the 3D model says are visible that have a pixel within one pixel of the projected position and within 0.1 of the eye color in OKLab; **color** is a symmetric, area-weighted chamfer distance ($\times 100$) in OKLab between the concept’s eight color clusters and the sprite’s colors.

Against the naive baseline (Table 3, Figure 5), the full pixelization reduces noise by 59%, nearly eliminates detached fragments and raises eye legibility from 53% to 92%. Cleanup lowers noise in all 14 creatures; projected eyes add 20 points of eye legibility; concept color lowers the color distance in all 14; thickening matters only where limbs are thin (the beetle’s detached fragments drop from 0.47 to 0.34 per frame). Feature protection slightly raises the noise measure because it deliberately keeps one-pixel highlights. The mushroom is the clearest failure: its cap hides the eyes, and only 6 of 32 survive.

Table 3: Pixelization ablation over 14 creatures (thickening: the 7 legged ones). Lower is better except for eyes.

Variant	Noise %	Fragm.	Eyes %	Color
Naive downscale	8.29	0.28	53.1	6.54
No cleanup	4.94	0.03	91.3	4.75
No projected eyes	3.27	0.03	71.9	4.50
No concept color	3.35	0.03	91.9	6.16
No feature protection	2.88	0.03	91.9	4.47
Full	3.36	0.03	91.9	4.40
No thickening (7 legged)	4.42	0.071	82.4	4.65
Full (same 7)	4.45	0.054	82.4	4.64



Figure 5: One frame per creature under each variant. The naive downscale blurs, loses outlines and breaks the beetle’s antenna; without concept color the serpent turns saturated red and the fox orange.

7.2 Prompt writer

FLUX.2 [klein] uses a complete Qwen3 4B language model as its text encoder, so a writer is available at no memory cost, but its translations are unreliable: it turned “zorro de hielo” (ice fox) into a snow wolf and “búho mago” (wizard owl) into a wise owl. The 27B writer translated all five test ideas correctly in 12–18 s, including loading (Figure 6).

7.3 Where single-image 3D spends its time

A lighter mesh is not faster: the cost lies in the network passes of the three flows (sparse structure 53 s, shape cascade 56 s, texture 65 s in the fast setting), not in the polygons. Reducing the cascade’s token budget has no effect, because it never drops below 1024^3 and the fox uses only 6,798 tokens. Six flow steps instead of eight, and no guidance in the two shape flows while keeping it in the structure flow, bring the step from 324 s to 203 s (Table 4)

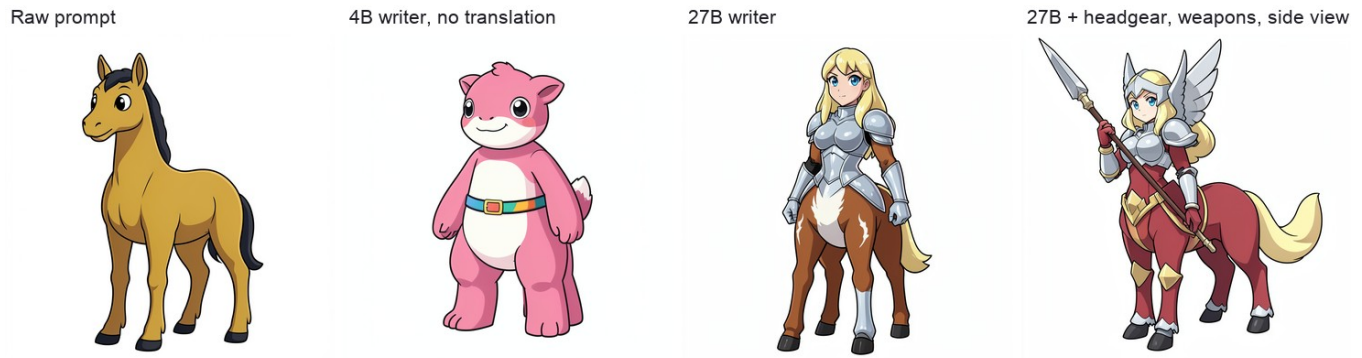


Figure 6: The same idea, “valkiria centaura” (Spanish for *valkyrie centaur*), four ways. Without a writer FLUX.2 [klein] draws a horse; a first writer based on the 4B text encoder FLUX already loads kept “Valkiria” as a name and produced a pink bear; the 27B writer produces an armored centaur; asking for headgear, weapons and a side view yields the intended character in all four seeds.

Table 4: 3D generation settings for the fox on the Apple M5.

Setting	Time	Result
Full (8 steps, guided, large textures)	324 s	reference
8 steps, token cap 16,384	313 s	same
6 steps	250 s	same at sprite scale
6 steps, no shape guidance	203 s	same at sprite scale
4 steps	88 s	fragmented mesh



Figure 7: Route B sheets dropped into a top-down HD-2D test scene built with three.js.

with equivalent meshes; the beetle kept its thin legs and antennae in 176 s. With the writer (12–18 s), the concept (15–38 s), rigging and 32 renders (20–60 s) and pixelization (5–15 s), a creature takes about four and a half minutes.

Table 5: The two routes side by side.

	Route A: learned 2D	Route B: 3D proxy
Hardware	NVIDIA RTX 3090	Apple M5
Training	≈44 GPU hours	none
Sprite data	2,984 sheets, non-commercial	none
New designs	memorizes after ≈30k steps	any idea, any language
Directions	5 generated + 3 mirrored	8 rendered
Animation	Walk and Idle, learned	Walk, rule-based, 13 body plans
Style	native, real palettes	approximated by pixelization
Best result	part assembly from real sprites	14 creatures, 12 body plans

7.4 Application

The sheets drop directly into a game scene (Figure 7). In our three.js [29] test scene each creature wanders on its own and picks the sheet row that matches its walking direction relative to the camera.

8 ROUTE A VERSUS ROUTE B

Table 5 sets the routes side by side. Route A draws in the native style of its training data, applies real palettes and has learned an Idle cycle; Route B does none of that. But Route A is bounded by its data: it needs a corpus of sheets under a non-commercial license, it memorizes its 2,603 designs, its best output reuses real parts, and mirroring rules out asymmetric characters. Route B needs no sprite data or training, inherits novelty from an open-domain image model, renders all eight directions from one geometry, and runs on a single consumer machine. We have no metric that applies to both, so this verdict is qualitative, but it is not close: Route B is the route that produces new, consistent characters today.

9 DISCUSSION

Art direction. We have not fine-tuned FLUX.2 to lock a single visual style or art direction; the concepts share a style only through the hidden template. Even so, splitting the problem by body plan, and covering as many body plans as possible with automatic rigging, works well in practice: the body plan fixes the pose the image model is asked for, the anatomy the writer spells out, how the mesh is straightened and how it walks.

Learned animation. A text-driven animation model for arbitrary skeletons, UniMate [9], was released on the day this draft was written. It has not been tested in this version. Combined with learned rigging [33], it could add the attack, hurt and sleep animations that games also need.

Toward a single model. Route A failed for lack of data, and Route B is a data generator. Each Route B creature yields a consistent eight-direction sheet that can be converted to pixel roles. Route A's palette donation multiplies each sheet by real palettes without inventing colors: from 2,602 donor palettes, the three creatures tested kept between 36 and 498 distinct color families after removing near-duplicates. Mirroring adds a reflected copy of each. Grouped by body plan, this oversampling could build the per-body-plan datasets Route A lacked, and the initial topology, a sheet U-Net over roles, could then learn to generate art with walk cycles directly. Today there are not enough training samples to do that without a pipeline.

10 LIMITATIONS

Evaluation. Route B's metrics are proxies on 14 self-generated creatures; we have run no perceptual study and no comparison against multi-view diffusion, and the two routes share no metric.

The proxy invents. The back of a creature is a guess: the ghost's back has blotches absent from the concept, and a serpent seen from the front collapses into a coil.

Animation. Only legs and, for some plans, arms move; tails and heads are static, the dragon plan is untested, and walking is the only action.

Licenses. Route B uses no sprite data, but Route A and any reference-based evaluation rely on community sprites licensed for non-commercial use only.

11 CONCLUSION

No single model we tried turns an idea into a consistent eight-direction walking sprite. A pipeline does. Of the two routes we built, learning sheets directly hit the limits of a few thousand designs, while a disposable 3D proxy, a rule-based rig per body plan and a pixelization that uses the 3D data produced new characters in every direction on one consumer machine. It is the better and the easier path today. Used as an oversampler together with palette donation and mirroring, it is also the most promising way to build the datasets a future single model will need.

ACKNOWLEDGMENTS

Both routes were developed with the assistance of Claude (Anthropic): Route A on the RTX 3090 machine, including a Claude Opus agent supervising training, and Route B and this draft on the Apple M5. All models ran locally.

REFERENCES

- [1] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. 2021. Structured Denoising Diffusion Models in Discrete State-Spaces. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [2] Ilya Baran and Jovan Popović. 2007. Automatic Rigging and Animation of 3D Characters. *ACM Transactions on Graphics (Proc. SIGGRAPH)* 26, 3 (2007).
- [3] James Betker, Gabriel Goh, Li Jing, Tim Brooks, Jianfeng Wang, Linjie Li, Long Ouyang, Juntang Zhuang, Joyce Lee, Yufei Guo, Wesam Manassra, Prafulla Dhariwal, Casey Chu, Yunxin Jiao, and Aditya Ramesh. 2023. *Improving Image Generation with Better Captions*. Technical Report. OpenAI.
- [4] BingelJell. 2026. Image-to-3D Lab. <https://github.com/BingelJell/image-to-3dlab>.
- [5] Alexandre Binniger and Olga Sorkine-Hornung. 2024. SD- π XL: Generating Low-Resolution Quantized Imagery via Score Distillation. In *SIGGRAPH Asia 2024 Conference Papers*.
- [6] Black Forest Labs. 2026. FLUX.2 [klein] 4B model card. <https://huggingface.co/black-forest-labs/FLUX.2-klein-4B>.
- [7] Blender Online Community. 2026. Blender – a 3D Modelling and Rendering Package. <https://www.blender.org>.
- [8] Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T. Freeman. 2022. MaskGIT: Masked Generative Image Transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [9] Friedrich-M. 2026. UniMate: Text-Driven Animation for Arbitrary Skeletons. <https://github.com/Friedrich-M/UniMate>.
- [10] Timothy Gerstner, Doug DeCarlo, Marc Alexa, Adam Finkelstein, Yotam Gingold, and Andrew Nealen. 2012. Pixelated Image Abstraction. In *Proceedings of the Symposium on Non-Photorealistic Animation and Rendering (NPAR)*.
- [11] Chu Han, Qiang Wen, Shengfeng He, Qianshu Zhu, Yinjie Tan, Guoqiang Han, and Tien-Tsin Wong. 2018. Deep Unsupervised Pixelization. *ACM Transactions on Graphics (Proc. SIGGRAPH Asia)* 37, 6 (2018).
- [12] Yaru Hao, Zewen Chi, Li Dong, and Furu Wei. 2023. Optimizing Prompts for Text-to-Image Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [13] Jonathan Ho and Tim Salimans. 2022. Classifier-Free Diffusion Guidance. *arXiv preprint arXiv:2207.12598* (2022).
- [14] Johannes Kopf and Dani Lischinski. 2011. Depixelizing Pixel Art. *ACM Transactions on Graphics (Proc. SIGGRAPH)* 30, 4 (2011).
- [15] Michiyoshi Kuwahara, Kozaburo Hachimura, Shigeru Eiho, and Masato Kinoshita. 1976. Processing of RI-Angiocardiographic Images. In *Digital Processing of Biomedical Images*. Springer, 187–202.
- [16] Tuomas Kynkäänniemi, Miika Aittala, Tero Karras, Samuli Laine, Timo Aila, and Jaakko Lehtinen. 2024. Applying Guidance in a Limited Interval Improves Sample and Distribution Quality in Diffusion Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [17] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. 2023. Flow Matching for Generative Modeling. In *International Conference on Learning Representations (ICLR)*.
- [18] Ruoshi Liu, Rundi Wu, Basile Van Hoorick, Pavel Tokmakov, Sergey Zakharov, and Carl Vondrick. 2023. Zero-1-to-3: Zero-shot One Image to 3D Object. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.
- [19] Junya Christopher Motomura. 2015. GuiltyGearXrd's Art Style: The X Factor Between 2D and 3D. Game Developers Conference (GDC).
- [20] Ollama. 2026. Ollama: Run Large Language Models Locally. <https://ollama.com>.
- [21] Björn Ottosson. 2020. A Perceptual Color Space for Image Processing. <https://bottosson.github.io/posts/oklab/>.
- [22] PMD Collab. 2026. PMD Sprite Repository. <https://sprites.pmdcollab.org>.
- [23] Dustin Podell, Zion English, Kyle Lacey, Andreas Blattmann, Tim Dockhorn, Jonas Müller, Joe Penna, and Robin Rombach. 2023. SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis. *arXiv preprint arXiv:2307.01952*.
- [24] Qwen Team. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).
- [25] Qwen Team. 2026. Qwen-Image. <https://huggingface.co/Qwen>.
- [26] raven38. 2026. Pixa3D single-view weights, Q8_0 GGUF. https://huggingface.co/raven38/pixa3d-sv-q8_0-v1.
- [27] Ruoxi Shi, Hansheng Chen, Zhuoyang Zhang, Minghua Liu, Chao Xu, Xinyue Wei, Linghao Chen, Chong Zeng, and Hao Su. 2023. Zero123++: a Single Image to Consistent Multi-view Diffusion Base Model. *arXiv preprint arXiv:2310.15110* (2023).

- [28] Tencent Hunyuan3D Team. 2025. Hunyuan3D 2.0: Scaling Diffusion Models for High Resolution Textured 3D Assets Generation. *arXiv preprint arXiv:2501.12202* (2025).
- [29] three.js authors. 2026. three.js: JavaScript 3D Library. <https://threejs.org>.
- [30] Thomas Vasseur. 2018. Art Design Deep Dive: Using a 3D Pipeline for 2D Animation in *Dead Cells*. Game Developer (Gamasutra).
- [31] Vikram Voleti, Chun-Han Yao, Mark Boss, Adam Letts, David Pankratz, Dmitry Tochilkin, Christian Laforte, Robin Rombach, and Varun Jampani. 2024. SV3D: Novel Multi-view Synthesis and 3D Generation from a Single Image using Latent Video Diffusion. In *European Conference on Computer Vision (ECCV)*.
- [32] Jianfeng Xiang, Zelong Lv, Sicheng Xu, Yu Deng, Ruicheng Wang, Bowen Zhang, Dong Chen, Xin Tong, and Jiaolong Yang. 2025. Structured 3D Latents for Scalable and Versatile 3D Generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [33] Jia-Peng Zhang, Cheng-Feng Pu, Meng-Hao Guo, Yan-Pei Cao, and Shi-Min Hu. 2025. One Model to Rig Them All: Diverse Skeleton Rigging with UniRig. *ACM Transactions on Graphics (Proc. SIGGRAPH)* 44, 4 (2025).